

“Process Termination Messages – A Linux Utility for Reporting Process Exit Status”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

A. SHASHANK REDDY	2520090139
D. VIGNESH KUMAR	2520090071
Y. SHARATH GOUD	2520090062

Under the Esteemed Guidance of

M. Raghupathi

Assistant Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “**Process Termination Messages – A Linux Utility for Reporting Process Exit Status**” is a record of Bonafide work of **A. SHASHANK REDDY - 2520090139, D. VIGNESH KUMAR - 2520090071, Y. SHARATH - 2520090062** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

A. SHASHANK REDDY	2520090139
D. VIGNESH KUMAR	2520090071
Y. SHARATH	2520090062

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled **“Process Termination Messages – A Linux Utility for Reporting Process Exit Status”** is a bonafide work done and submitted by **A. Shashank Reddy – 2520090139, D. Vignesh Kumar – 2520090071, Y. Sharath – 2520090062**, in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator [**Course Coordinator Name**], and my guide [**Guide Name**], Department of Computer Science and Information Technology which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to [**HOD Name**] (CSIT), Head of the Department of Computer Science and Information Technology for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

A. SHASHANK REDDY 2520090139

D. VIGNESH KUMAR 2520090071

Y. SHARATH 2520090062

Abstract

Process termination in Linux can occur in several ways — normal exit with a status code, termination by an unhandled signal, or abnormal exit with a core dump — yet most simple programs report little of this to the user. This project, Process Termination Messages, builds a Linux utility that creates child processes using `fork()` and `execvp()`, monitors their completion with `wait()/waitpid()`, and decodes the returned status using the POSIX macros `WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED`, `WTERMSIG` and `WCOREDUMP`.

The utility prints clear, human-readable termination messages reporting the process ID, the command executed, the exit code or terminating signal, and whether a core dump was produced. It also demonstrates asynchronous notification through the `SIGCHLD` signal handled via `sigaction()`, reinforcing core Operating Systems concepts such as process creation, the parent-child relationship, and kernel-level exit-status communication. The expected outcome is a working command-line tool that launches and tracks processes, giving practical experience with POSIX process-management system calls in Linux.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1–5
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background		6–12
		2.1 Operating System Concepts Related to the	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		13–17
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		18–22
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	

5	System Architecture and Design		23–29
		5.1 Overall System Architecture	
		5.2 System Workflow	
		5.3 Flowchart / Data Flow	
		5.4 Module Description	
		5.5 Input and Output Design	
6	Implementation		30–45
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	Execution and Demonstration		62–67
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	Conclusion		79–83
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	References		84–86

		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

An operating system manages every running program as a process. On Linux, new processes are created by an existing process: the parent calls `fork()` to make a copy of itself, and the child then calls one of the `exec` functions, such as `execvp()`, to replace its memory image with a different program. The parent and child form a relationship that lasts until the child terminates and the parent collects its termination information using `wait()` or `waitpid()`.

A process can terminate in more than one way. It may finish normally and return an exit code, it may be stopped by a signal that it does not handle (for example `SIGSEGV` after an invalid memory access, or `SIGKILL` sent by another process), or it may be terminated by a signal that also produces a core dump for later debugging. The kernel records which of these happened in a single integer called the wait status, and the parent must decode this integer with the POSIX macros `WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED`, `WTERMSIG` and `WCOREDUMP`.

Most simple programs and shell scripts only check whether the exit code is zero or non-zero. As a result, the exact cause of termination is often hidden from the user. This project, Process Termination Messages, builds a Linux command-line utility that launches programs, monitors their termination and reports exactly how each one ended.

1.2 Problem Statement

Standard Linux shells and simple programs typically report only whether a child process succeeded or failed. They do not clearly distinguish between a normal exit, termination by a signal such as `SIGSEGV` or `SIGKILL`, and an abnormal exit that produced a core dump. Developers need this finer-grained information to debug crashes and to build robust process-supervision tools, but correctly decoding the `wait()` status word using the POSIX macros is non-trivial for beginners.

There is therefore a need for a reliable Linux utility that accurately captures and reports the exact cause and manner of process termination in a clear, human-readable form.

1.3 Scope of the Project

The scope of this project is limited to process creation, monitoring and termination reporting on a Linux/Ubuntu system, implemented in C using POSIX system calls. The project covers the following:

- Creating one or more child processes using `fork()` and running a user-specified command in each child using `execvp()`.
- Waiting for children synchronously using `wait()/waitpid()`, and asynchronously using a `SIGCHLD` handler registered with `sigaction()`.
- Decoding the wait status to report a normal exit with its exit code, termination by a signal with the signal name, and whether a core dump was produced.
- Printing a formatted message containing the process ID, the command, a timestamp and the outcome, and producing sample logs for both normal and abnormal exits.

The project does not aim to replace a full process-management system such as `systemd`, and it does not include graphical interfaces, remote monitoring or automatic restarting of failed processes.

1.4 Objectives of the Project

- To design a parent process that creates one or more child processes using `fork()` and executes target programs using `execvp()`.
- To monitor child termination using `wait()` and `waitpid()`, and to decode the status using `WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED`, `WTERMSIG` and `WCOREDUMP`.
- To generate formatted, human-readable termination messages reporting the PID, command name, exit code or signal, and core-dump status.
- To handle asynchronous termination notifications for multiple concurrent child processes using the `SIGCHLD` signal and a `sigaction()`-based handler.

1.5 Importance of the Application

Knowing exactly why a process ended is essential for debugging. A program that exits with code 1 has reported an error of its own, whereas a program killed by `SIGSEGV` has crashed because of a memory-access fault, and a program killed by `SIGKILL` was stopped from outside. Each case needs a different response, yet a plain success/failure result cannot tell them apart.

The utility also has strong educational value. It brings together several core Operating Systems concepts in one working tool: process creation, the parent-child relationship, program replacement, zombie prevention through proper waiting, signals, and kernel-to-user communication of exit status. Building it gives hands-on experience with the POSIX process-management system calls.

1.6 Target Users / Audience

- **Students and educators** learning Operating Systems and systems programming, who can use the tool to observe process lifecycle behaviour directly.
- **Software developers** who need to find out why a program crashed or was terminated during development and testing.
- **Test and QA engineers** who run programs in bulk and need a clear record of how each one ended.
- **System administrators** who want a simple, lightweight way to supervise and log the termination of commands and scripts.

CHAPTER -2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

Process and process creation. A process is a program in execution, together with its own address space, process ID and kernel bookkeeping data. In Linux a new process is created with `fork()`, which returns twice: it returns the child's PID to the parent and 0 to the child. The two processes then run independently, which is why the program must test the return value to decide which code each should run.

Program replacement with `exec`. The `exec` family of functions loads a new program into the calling process. `execvp()` searches the directories listed in the `PATH` environment variable for the command and runs it with the given argument vector. If it succeeds it never returns, so any code after it runs only when the command could not be started.

Waiting and zombie processes. When a child terminates, the kernel keeps its exit information until the parent reads it with `wait()` or `waitpid()`. Until then the child remains as a zombie entry in the process table. A parent that never waits leaves zombies behind, so correct waiting is part of correct process management. If a parent terminates first, its children become orphans and are adopted by the `init` process.

The wait status word. `wait()` and `waitpid()` return the child's termination information as an integer that packs several facts together. The POSIX macros listed in Table 2.1 are the portable way to read it, and they are used throughout this project.

Table 2.1: POSIX macros for decoding the `wait()` status

Macro	Meaning
<code>WIFEXITED(status)</code>	True if the child terminated normally, by calling <code>exit()</code> or returning from <code>main()</code> .
<code>WEXITSTATUS(status)</code>	The exit code of the child (the low 8 bits passed to <code>exit()</code>). Valid only if <code>WIFEXITED</code> is true.
<code>WIFSIGNALED(status)</code>	True if the child was terminated by a signal.
<code>WTERMSIG(status)</code>	The number of the signal that terminated the child. Valid only if <code>WIFSIGNALED</code> is true.
<code>WCOREDUMP(status)</code>	True if the terminating signal produced a core dump. Valid only if <code>WIFSIGNALED</code> is true.

Signals and `SIGCHLD`. A signal is a software notification delivered by the kernel to a

process. Some signals, such as SIGSEGV (invalid memory reference), SIGABRT, SIGFPE and SIGKILL, terminate a process, and some of them also cause a core dump. Whenever a child process terminates, the kernel sends SIGCHLD to its parent. By installing a handler for SIGCHLD with `sigaction()`, the parent can be told about a termination as it happens instead of blocking in `wait()`.

Core dumps. A core dump is a file containing the memory image of a process at the moment it was terminated by certain signals. Debuggers such as GDB can load it to inspect the state of the program at the point of the crash. The `WCOREDUMP` macro tells the parent whether such a dump was produced.

Supporting calls. `strsignal()` converts a signal number into a readable description such as "Segmentation fault", and `getpid()` and `getppid()` return the process ID of the caller and of its parent, which are needed for identifying processes in the output messages.

2.2 Existing Methods / Algorithms

Several existing tools report some information about how a process ended, but none of them is a simple, dedicated utility for this purpose:

- **Shell exit status (\$?).** The shell stores the exit status of the last command. For a process killed by a signal, the shell reports the value 128 plus the signal number (for example 139 for SIGSEGV). This is only a convention and it cannot be told apart from a program that really called `exit(139)`, and it gives no core-dump information.
- **Shell job messages.** Interactive shells such as bash print messages like "Segmentation fault (core dumped)" for some foreground jobs. These messages are shown only by the shell, are not available for use in a program, and do not include the PID, command and timestamp in a structured form.
- **strace.** The `strace` tool traces system calls and prints how the traced process ended. It gives detailed information but produces a very large amount of output and slows the program, so it is too heavy for simple termination reporting.
- **Process supervisors (systemd, supervisord).** These tools record why a service stopped, but they are large systems intended for managing long-running services and are not suitable as a small learning tool.
- **Hand-written wait() code.** Programs that call `wait()` directly often check only the exit code or ignore the status altogether, because decoding the status correctly with the macros is easy to get wrong.

2.3 Proposed Approach

The proposed system is a C program for Linux/Ubuntu in which a parent process launches and monitors commands. It works in the following steps:

- The parent reads the command (or commands) to be run from the command line.
- For each command the parent calls `fork()`. In the child, `execvp()` replaces the process image with the requested program; if `execvp()` fails, the child prints an error and exits with a non-zero code so that the failure is also reported.
- In the synchronous mode, the parent waits for children in a loop using `waitpid(-1, &status, 0)` until none remain.
- In the asynchronous mode, the parent registers a `SIGCHLD` handler with `sigaction()`. When the kernel delivers the signal, the handler collects every finished child with `waitpid(-1, &status, WNOHANG)`, so that several children that end at nearly the same time are all reported and none is left as a zombie.
- For each collected status, the program uses `WIFEXITED` and `WEXITSTATUS` for a normal exit, or `WIFSIGNALED`, `WTERMSIG` and `WCOREDUMP` for a signal termination, converts the signal number to a name with `strsignal()`, and prints a formatted message with the PID, command, timestamp and outcome.

The handler is written to keep its work minimal, since only a limited set of functions is safe to call inside a signal handler. The format of the messages is illustrated below (the values are examples only):

```
[2026-10-07 10:15:32] PID 4321 | ls -l | Exited normally with exit code 0
[2026-10-07 10:15:33] PID 4322 | ./crash | Terminated by signal 11
(Segmentation fault), core dumped
[2026-10-07 10:15:35] PID 4323 | sleep 100 | Terminated by signal 9
(Killed), no core dump
```

The utility will be tested with processes that exit normally, that are killed by signals such as `SIGSEGV` and `SIGKILL`, and that dump core, to check that the correct message is produced in every case.

2.4 Advantages of the Proposed Approach

- **Accurate and complete reporting.** Normal exits, signal terminations and core dumps are each identified and reported separately, using the standard POSIX macros instead of guesses based on the value of `$?` .
- **Clear output.** Every message contains the PID, command, timestamp and outcome in a readable format, which makes crashes easy to identify and logs easy to read.

- **Handles multiple processes.** The SIGCHLD-based design reports several concurrent children as soon as each terminates, without blocking the parent.
- **No zombie processes.** Every terminated child is collected with waitpid(), so no zombie entries are left in the process table.
- **Lightweight and portable.** The tool uses only standard C and POSIX system calls, needs no extra libraries, and runs on any Linux system with GCC.
- **Educational value.** The implementation demonstrates process creation, exec, waiting, signals and exit-status communication in a single, compact program.